🚀 MicroMove (Mimo_v1) 团队协同与多端交付开发手册

> **文档版本**：v1.0.0 (Release)
> **适用对象**：Master（架构/App 负责人）、Web 前端开发、Claude/Codex 插件开发
> **统一数据中枢**：Supabase (`https://cdpgjujgovomsqsgxzqz.supabase.co`)
> **GitHub 代码仓库**： `https://github.com/Gozoyo/Mimo_v1.git`

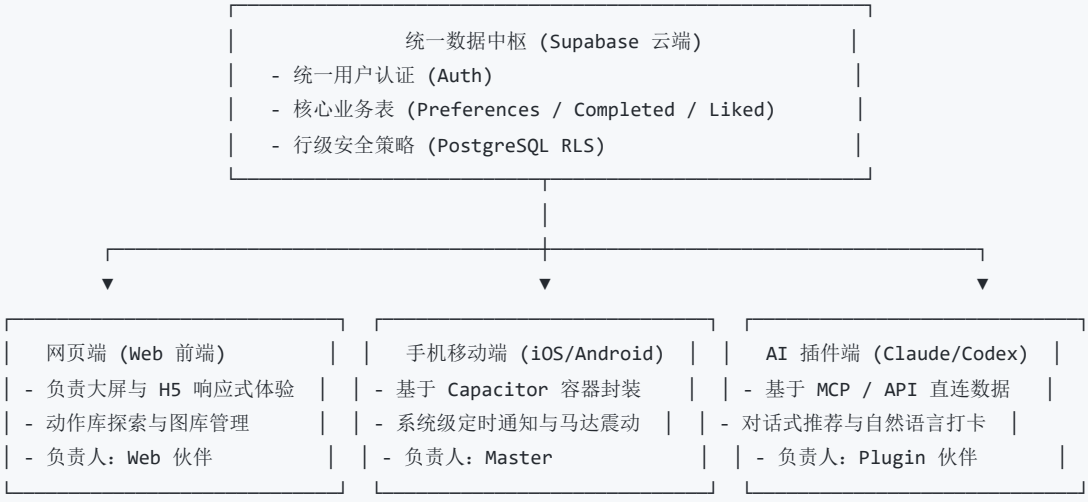
目录

1. 项目全景架构与团队分工
2. 从 GitHub 拉取并在本地运行指南（全员必读）
3. AI 插件端 (Codex / Claude Plugin / MCP) 接入手册
4. Master 移动端打包指南 (Android APK 与 iOS 发布)
5. 常见问题与技术避坑指南 (FAQ)

---

1. 项目全景架构与团队分工

MicroMove 定位为**工位久坐场景下的轻量打断与习惯养成微运动工具**，采用 **「一套核心业务数据 + 多端统一协同」** 架构：



团队职责边界明确

- **Master (你)**：维护 GitHub 主仓库、统筹 Supabase 数据结构升级、使用 Capacitor 打包 Android APK 与 iOS 安装包。
- **Web 同伴**：专注于界面美化、桌面大屏适配、个人中心与历史足迹等功能开发。

- **Plugin 同伴**：编写 Claude Plugin / MCP Server / Codex 工具，利用 Supabase API 实现"对话即打卡、对话即推荐"。

---

2. 从 GitHub 拉取并在本地运行指南（全员必读）

任何新成员接手代码，按以下步骤即可在 3 分钟内本地启动：

2.1 环境准备

- **Node.js**：建议安装 Node.js 18.x 或 20.x+（官网下载 LTS 版本）；
- **Git**：安装 Git 命令行工具。

2.2 克隆与依赖安装

打开终端（Windows Terminal / CMD / macOS Terminal），执行：

```
# 1. 从 GitHub 克隆项目到本地
git clone https://github.com/Gozoyo/Mimo_v1.git

# 2. 进入项目主目录
cd Mimo_v1

# 3. 安装全量依赖包
npm install
```

2.3 配置云端环境变量

在 `Mimo_v1` 根目录下，新建一个 `.env.local` 文件（注意前缀有点），粘贴以下统一配置：

```
# Supabase 项目基础 API 地址
VITE_SUPABASE_URL=https://cdpgjujgovomsqsgxzqz.supabase.co

# 客户端只读/受限公钥 (受数据库 RLS 行级安全保护)
VITE_SUPABASE_ANON_KEY=sb_publishable_BBFoLf-tOardisfL_YRYLw_m3ZUK1R3
```

2.4 极速启动与体验

- **方式 1（命令行）**：
  `bash npm run dev`
  在浏览器中访问：`http://localhost:5173/`。
- **方式 2（Windows 一键直达）**：
  直接双击目录下的 `启动Mimo.bat`，脚本会自动启动服务并自动弹出浏览器！

2.5 代码修改与推送 (Git 工作流)

建议成员在独立分支上开发，避免冲突：

```
# 1. 切换新功能分支
git checkout -b feat/web-enhancement

# 2. 提交修改
git add .
git commit -m "feat: 完善动作库筛选交互"

# 3. 推送到远程
git push origin feat/web-enhancement
```

提示：如果是首次向 GitHub 推送，Git 会在浏览器中弹出授权窗口，点击「Authorize」一键授权即可。

---

3. AI 插件端 (Codex / Claude Plugin / MCP) 接入手册

AI 插件不需要运行前端界面，核心任务是：**接收用户的自然语言指令 ➔ 调用工具函数 (Tool Call) ➔ 直连 Supabase 读写数据。**

3.1 连接基础信息

- **Project URL**: `https://cdpgjujgovomsqsgxzqz.supabase.co`
- **Anon Key**: `sb_publishable_BBFoLf-tOardisfL_YRYLw_m3ZUK1R3`

3.2 方案 A：TypeScript / Node.js 版 (适合 MCP Server / Vercel AI SDK)

安装官方 SDK：

```
npm install @supabase/supabase-js
```

核心工具函数实现：

```
import { createClient } from '@supabase/supabase-js';

const SUPABASE_URL = 'https://cdpgjujgovomsqsgxzqz.supabase.co';
const SUPABASE_ANON_KEY = 'sb_publishable_BBFoLf-tOardisfL_YRYLw_m3ZUK1R3';

export const supabase = createClient(SUPABASE_URL, SUPABASE_ANON_KEY);

/**
 * 工具 1：根据用户 ID 获取该用户的微运动习惯偏好
 */
export async function getPreferencesTool(userId: string) {
  const { data, error } = await supabase
    .from('preferences')
    .select('*')
    .eq('user_id', userId)
    .single();

  if (error) throw error;
  return data;
}

/**
 * 工具 2：用户在对话中完成拉伸动作后，调用此函数记录打卡
 */
export async function recordSessionTool(params: {
  userId: string;
  exerciseId: string;       // 动作 ID，例如 "neck_side_tilt_left"
  targetArea: string;       // 部位，例如 "肩颈"
  durationSec: number;      // 实际拉伸秒数，例如 30
}) {
  const today = new Date().toISOString().split('T')[0];

  const { data, error } = await supabase
    .from('completed_sessions')
    .insert({
      id: crypto.randomUUID(),
      user_id: params.userId,
      source: 'claude_plugin', // 标记来源为 AI 插件
      completed_at: new Date().toISOString(),
      date: today,
      planned_duration_sec: params.durationSec,
      actual_completed_sec: params.durationSec,
      movement_count: 1,
      target_areas: [params.targetArea],
      exercise_ids: [params.exerciseId]
    });

  if (error) throw error;
  return { success: true, message: `打卡已记录！今日身体电量已同步充能。` };
}
```

3.3 方案 B：Python 版 (适合 LangChain / FastAPI / Python MCP)

安装依赖：

```
pip install supabase
```

Python 实现：

```
import datetime
import uuid
from supabase import create_client, Client

SUPABASE_URL = "https://cdpgjujgovomsqsgxzqz.supabase.co"
SUPABASE_KEY = "sb_publishable_BBFoLf-tOardisfL_YRYLw_m3ZUK1R3"

supabase: Client = create_client(SUPABASE_URL, SUPABASE_KEY)

def record_session(user_id: str, exercise_id: str, target_area: str, seconds: int = 30):
    """记录用户微运动打卡并同步到云端"""
    today_str = datetime.date.today().isoformat()
    payload = {
        "id": str(uuid.uuid4()),
        "user_id": user_id,
        "source": "claude_plugin",
        "completed_at": datetime.datetime.utcnow().isoformat(),
        "date": today_str,
        "planned_duration_sec": seconds,
        "actual_completed_sec": seconds,
        "movement_count": 1,
        "target_areas": [target_area],
        "exercise_ids": [exercise_id]
    }
    res = supabase.table("completed_sessions").insert(payload).execute()
    return res.data
```

3.4 关键数据表字段字典（严禁私自修改）

数据表名	关键字段	类型	说明与合法取值
`preferences`	`user_id`	UUID	关联 auth.users 用户主键
	`duration_min`	Integer	单次偏好时长（分钟，例如 2）
	`session_duration_sec`	Integer	单次偏好时长（秒，例如 150）
	`target_areas`	Text[]	`["肩颈", "上背", "胸肩", "腰背"]` 组合
	`space`	Text	`"小空间" | "中空间" | "大空间"`
	`intensity`	Text	`"温和" | "中等"`

数据表名	关键字段	类型	说明与合法取值
`completed_sessions`	`id`	UUID	记录唯一主键
	`user_id`	UUID	关联用户 ID
	`source`	Text	固定填 `"claude_plugin"` 或 `"codex_plugin"`
	`date`	Date	日期格式 `YYYY-MM-DD`
	`actual_completed_sec`	Integer	实际完成秒数（如 30）
	`exercise_ids`	Text[]	动作标准 ID（如 `["neck_side_tilt_left"]`）

---

4. Master 移动端打包指南 (Android APK 与 iOS 发布)

项目已内置 **Capacitor 8** 原生引擎。我们写的一套 React 代码，能够同时在 Android 和 iOS 原生容器中直接运行，并调用手机震动马达和本地推送通知。

4.1 出包前提：编译并同步原生工程

每当前端代码（`src/`）有改动后，打包前必须执行两行命令：

```
# 1. 编译生成 Web 生产产物至 dist/ 目录
npm run build

# 2. 将产物同步至 android/ 和 ios/ 原生工程
npx cap sync
```

4.2 打包 Android (生成 APK 安装包)

步骤 1：启动 Android Studio

在项目根目录运行命令：

```
npx cap open android
```

系统会自动启动 Android Studio 并加载 `android/` 工程。

步骤 2：配置应用信息（如需定制）

- **应用名**：在 `android/app/src/main/res/values/strings.xml` 中修改 `<string name="app_name">MicroMove</string>`；
- **图标替换**：右键点击 `app/res` ➔ `New` ➔ `Image Asset`，选择图片生成自适应应用图标。

步骤 3：一键构建 APK

1. 在 Android Studio 顶部菜单栏点击：`Build` ➔ `Build Bundle(s) / APK(s)` ➔ `Build APK(s)`；
2. 编译约 1~2 分钟，右下角弹出提示 `Build APK: Locate`；
3. 点击 `Locate`，即可在 `android/app/build/outputs/apk/debug/` 下拿到 `app-debug.apk`；
4. 将该 `.apk` 文件发送到安卓手机上直接安装运行！

4.3 打包 iOS (生成苹果 App / TestFlight)

注：打包 iOS 需要在 macOS 环境下，并安装有 Xcode。

步骤 1：打开 Xcode 工程

```
npx cap open ios
```

步骤 2：配置开发者签名

1. 在 Xcode 左侧点击根节点 `App`；
2. 进入 `Signing & Capabilities` 选项卡；
3. 勾选 `Automatically manage signing`，在 `Team` 下选择你的 Apple Developer 账号。

步骤 3：真机调试与分发

- **真机调试**：使用数据线连接 iPhone，在 Xcode 顶部设备栏选中你的真机，点击 `Run`（▶ 按钮）即可秒级将 App 安装到手机上。
- **分发测试**：顶部菜单选择 `Product` ➔ `Archive`，归档完成后点击 `Distribute App` 即可一键上传至 TestFlight 进行内部测试分发。

---

5. 常见问题与技术避坑指南 (FAQ)

Q1: 连续打卡或者数据为什么在两台电脑/手机上能看到同一份?

答：我们在 `src/storage.ts` 中实现了本地秒开与 Supabase 云端全量双向同步。打卡时自动推送到 `completed_sessions` 表；登录时自动拉取。只要凭同一账号登录，数据 100% 互通。

Q2: 为什么有时接口报 521 / Could not resolve host?

答：这是 Supabase 免费版的"7天自动休眠机制"。如果连续 7 天没有 API 请求，容器会自动休眠。
唤醒方法：登录 Supabase 控制台，在项目卡片上点击 **「Restore project」**，等待 1 分钟即可全量恢复，数据绝不丢失。

Q3: 团队成员在本地运行 `npm run dev` 报端口占用怎么办?

答：Vite 会自动递增尝试 `5174` 或 `5175` 端口，在终端中查看最终输出的绿色链接即可。

Q4: **动作库新增了动作，图片和动画放哪里？**

答：严格按照动作的英文 ID 命名并放入指定目录：
* 封面：`public/assets/exercises/covers/{id}.png`
* 演示图：`public/assets/exercises/demos/{id}.png`
* 动画：`public/assets/exercises/animations/{id}.json` 或 `.gif`
放入后无需修改代码，系统内置的 ID 解析引擎会自动完成 100% 映射与加载。

> 💡 **总结**：遵循本手册规范，Web 同伴、AI 插件同伴与 Master 可以完全解耦并行开发，零阻塞快速推进！